

Retinal OCT Image Classification with CNNs

Guide for Optics Students

Machine Learning for Optics

Abstract

This guide accompanies the Retinal OCT notebook exercise. It covers the physics of Optical Coherence Tomography, the retinal pathologies in the dataset, CNN fundamentals, and a step-by-step walkthrough of the solution.

Contents

1	Optical Coherence Tomography	2
1.1	Principle of Operation	2
1.2	Spectral-Domain OCT (SD-OCT)	2
2	Retinal Pathologies	2
3	Python Packages	2
4	Convolutional Neural Networks (CNNs)	2
4.1	Why CNNs for Images?	2
4.2	Building Blocks	3
4.3	Architecture Used	3
5	Handling Class Imbalance	3
6	Step-by-Step Solution	4
6.1	Step 1: Data Transforms	4
6.2	Step 2: Class Weights	4
6.3	Step 3: CNN Architecture	4
6.4	Step 4: Loss, Optimiser, Scheduler	4
6.5	Step 5: Evaluation Loop	5
6.6	Step 6: Grad-CAM	5
7	Common Mistakes	5
8	Further Exercises	5

1 Optical Coherence Tomography

1.1 Principle of Operation

OCT is a non-invasive, interferometric imaging technique that produces cross-sectional (B-scan) images of biological tissue with micrometre-scale axial resolution.

Key Optics Concepts

- **Low-coherence interferometry.** A broadband source ($\lambda \approx 840$ nm, bandwidth $\Delta\lambda \sim 50$ nm) is split into reference and sample arms. Interference occurs only when path lengths match within the coherence length.
- **Coherence length.** $l_c = \frac{2 \ln 2}{\pi} \frac{\lambda^2}{\Delta\lambda} \approx 6 \mu\text{m}$.
- **Axial resolution** $\Delta z \approx l_c$ (set by source bandwidth, independent of optics).
- **Lateral resolution** $\Delta x \approx 1.22 \lambda f/D$ (diffraction limit).

1.2 Spectral-Domain OCT (SD-OCT)

Modern clinical OCT uses spectral-domain detection: a spectrometer captures the full interference spectrum in a single shot. A Fourier transform recovers the depth profile (A-scan), and lateral scanning builds a 2-D B-scan.

2 Retinal Pathologies

The Kermayn et al. (2018) dataset contains four classes of retinal B-scans:

Class	Pathology	OCT Appearance
CNV	Choroidal neovascularisation	Abnormal blood vessels under the retina; sub-retinal fluid, disrupted outer layers
DME	Diabetic macular edema	Intraretinal fluid (cystoid spaces), retinal thickening
DRUSEN	Early AMD marker	Small bright deposits between RPE and Bruch's membrane
NORMAL	Healthy retina	Well-defined retinal layers, no fluid or deposits

Table 1: Four diagnostic classes in the OCT dataset.

Clinical Note

This exercise is for educational purposes. A real diagnostic system requires regulatory approval, validation on multi-device datasets, and clinical oversight.

3 Python Packages

4 Convolutional Neural Networks (CNNs)

4.1 Why CNNs for Images?

For 1-D data such as spectra, a multi-layer perceptron (MLP) suffices because features sit at fixed positions. Images are different:

- Pathological features can appear *anywhere* in the image.

Package	Role in this exercise
<code>torch</code>	Neural network framework (tensors, autograd, <code>nn</code>)
<code>torchvision</code>	Image transforms, <code>ImageFolder</code> dataset loader, pre-trained models
<code>kagglehub</code>	Download the Kermany OCT dataset from Kaggle
<code>numpy</code>	Array operations for class-weight computation
<code>matplotlib</code>	Plotting images, training curves, confusion matrices
<code>sklearn.metrics</code>	<code>confusion_matrix</code> , <code>classification_report</code>

Table 2: Required Python packages.

- A fully connected network on 128×128 pixels would have $\sim 16\,000$ input neurons, leading to millions of parameters and severe overfitting.

CNNs solve both problems through **weight sharing** (the same small filter slides across the image) and **pooling** (progressive spatial reduction).

4.2 Building Blocks

Conv2d($C_{\text{in}}, C_{\text{out}}, k, \text{padding}$) Applies C_{out} learnable $k \times k$ filters. With `padding=1` and $k = 3$, the spatial size is preserved.

BatchNorm2d Normalises each feature map to zero mean and unit variance, stabilising training.

ReLU $f(x) = \max(0, x)$. Non-linearity that enables the network to learn complex decision boundaries.

MaxPool2d(2) Halves the spatial dimensions, reducing computation and increasing the receptive field.

AdaptiveAvgPool2d((1,1)) Averages each feature map to a single value, producing a fixed-length vector regardless of input size.

Dropout(p) During training, randomly zeroes p fraction of neurons. Reduces overfitting.

Linear($d_{\text{in}}, d_{\text{out}}$) Fully connected layer mapping to class logits.

4.3 Architecture Used

Input (1, 128, 128)

```
-> Conv(1,32,3)+BN+ReLU+MaxPool -> (32, 64, 64)
-> Conv(32,64,3)+BN+ReLU+MaxPool -> (64, 32, 32)
-> Conv(64,128,3)+BN+ReLU+MaxPool -> (128, 16, 16)
-> Conv(128,256,3)+BN+ReLU+MaxPool-> (256, 8, 8)
-> AdaptiveAvgPool(1,1) -> (256,)
-> Dropout(0.5) -> Linear(256, 4)
```

Total trainable parameters: $\sim 400\,000$ (much fewer than a fully connected network on the same input).

5 Handling Class Imbalance

Medical datasets are rarely balanced. If 80% of samples are class A, a model that always predicts A achieves 80% accuracy while being clinically useless.

Weighted cross-entropy assigns each class a weight inversely proportional to its frequency:

$$w_c = \frac{N_{\text{total}}}{C \cdot N_c}$$

This ensures that a mistake on a rare class contributes more to the loss than one on a common class.

6 Step-by-Step Solution

6.1 Step 1: Data Transforms

Key Decision

We apply augmentation (flips, rotation, translation) *only* to the training set. The test set uses deterministic transforms so that evaluation is reproducible.

```
train_transform = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((128, 128)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.RandomAffine(degrees=0, translate=(0.05, 0.05)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5]),
])
```

Why **Grayscale**? OCT images are inherently greyscale, but some files may be saved as RGB. This ensures a single input channel.

Why **Normalize([0.5], [0.5])**? Scales pixel values from $[0, 1]$ to $[-1, 1]$, centring the distribution around zero.

6.2 Step 2: Class Weights

```
class_counts = np.bincount(targets, minlength=NUM_CLASSES)
class_weights = len(targets) / (NUM_CLASSES * class_counts)
```

6.3 Step 3: CNN Architecture

Use `nn.Sequential` to chain the four convolutional blocks:

```
self.features = nn.Sequential(
    nn.Conv2d(1, 32, 3, padding=1), nn.BatchNorm2d(32),
    nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64),
    nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(64, 128, 3, padding=1), nn.BatchNorm2d(128),
    nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(128, 256, 3, padding=1), nn.BatchNorm2d(256),
    nn.ReLU(), nn.MaxPool2d(2),
)
```

6.4 Step 4: Loss, Optimiser, Scheduler

```
criterion = nn.CrossEntropyLoss(weight=class_weights)
optimiser = torch.optim.Adam(model.parameters(), lr=1e-3)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimiser, T_max=EPOCHS)
```

Cosine annealing smoothly reduces the learning rate from its initial value to near zero, which helps the model converge to a sharper minimum in the final epochs.

6.5 Step 5: Evaluation Loop

```
with torch.no_grad():
    for xb, yb in test_dl:
        xb = xb.to(device)
        preds = model(xb).argmax(1).cpu()
        all_preds.append(preds)
        all_labels.append(yb)
```

Remember to call `model.eval()` first to disable dropout and use running statistics in batch normalisation.

6.6 Step 6: Grad-CAM

The three key parts:

1. **Forward hook:** capture the feature maps A^k from the last convolutional layer. Call `out.requires_grad_(True)` so gradients flow through the hook.
2. **Backward hook:** capture the gradients $\partial y^c / \partial A^k$.
3. **Compute CAM:** $\alpha_k = \text{mean}(\text{grads})$, then $L = \text{ReLU}(\sum_k \alpha_k A^k)$, upsampled to input size.

Common Pitfall

If the model was trained from scratch (no pretrained weights), the feature map output does not have `requires_grad=True` by default. You must call `out.requires_grad_(True)` inside the forward hook before registering the backward hook.

7 Common Mistakes

Mistakes to Avoid

1. **Applying augmentation to test data.** Test transforms should be deterministic.
2. **Forgetting `model.eval()`.** Dropout and batch-norm behave differently at test time.
3. **Ignoring class imbalance.** Without weighted loss, the model may achieve high accuracy by ignoring minority classes.
4. **Not saving the best model.** Always checkpoint based on validation accuracy, not training loss.
5. **Grad-CAM on wrong layer.** Hook the last convolutional layer, not a pooling or linear layer.
6. **Missing `requires_grad_(True)`.** Required for Grad-CAM when the model has no pre-trained weights.
7. **Using vertical flips.** Retinal layers have a top-to-bottom anatomical order; vertical flips produce unrealistic images.

8 Further Exercises

1. **Transfer learning.** Replace the custom CNN with a pretrained ResNet-18. Adapt the first convolution (`nn.Conv2d(1, 64, 7, stride=2, padding=3)`) and the final linear layer. Compare accuracy and convergence speed.
2. **Resolution study.** Retrain with `IMG_SIZE = 224`. Record accuracy and wall-clock time. Is the improvement worth the cost?
3. **Error analysis.** Find the test images with the highest softmax confidence for the wrong class. Overlay Grad-CAM. Are these borderline cases between two pathologies?

4. **Domain shift.** The dataset comes from a single OCT device. Discuss what happens when you deploy the model on scans from a different manufacturer (different noise profile, resolution, contrast). Research domain adaptation methods such as DANN or style transfer.

References

- D. Kermany et al., “Identifying Medical Diagnoses and Treatable Diseases by Image-Based Deep Learning,” *Cell*, vol. 172, no. 5, pp. 1122–1131, 2018.
- R. R. Selvaraju et al., “Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization,” *ICCV*, 2017.
- W. Drexler and J. G. Fujimoto, *Optical Coherence Tomography: Technology and Applications*, Springer, 2015.